We call `pushDown(pairs, 0)` to fix the third heap's property; helper function `firstLeafIndex` at line #2 will return 3 (since [7], the last position in the array, is a child of the node at index [2], which is thus the last internal node) and so we will enter the loop.

The children of element [0] are at positions [1], [2] and [3], and among them we'll choose the highest priority child of the root, at line #3; turns out it is the element `<Unencrypted password on DB, 10>`; its priority is larger than current element's, so at line #5 we will swap it with current element.

Overall, after one iteration of the while loop, we have the situation illustrated by sub-figure 2.8.B, and `currentIndex` is set to 1.

Therefore, when we enter the loop for the second time, at line #2 `currentIndex` evaluates to 1, which is still smaller than 3, the index of the first leaf. The children of element at index [1] are at indices [4], [5] and [6], and among those elements we look for the node with highest priority among current node's children, at line #3 in listing 2.4: turn out it is the element at index [4], `<Page loads take 2+ seconds, 7>`.

Therefore, at line #4, we find out that, this time, all children's priority are all smaller than current element's, and we break out of the loop and return without any further change.

Listing 2.5 An optimization of the pushDown method

```
function pushDown(pairs, index=0)
```